

PresentationAgent

智能工作区平台

基于 AI Agent 的智能工作区平台，将复杂 AI 能力封装为简单用户交互

Web Deck

智能演示文稿生成

Draw.io

流程图智能编排

Web Sandbox

交互式Web沙盒

智能文档

Markdown技术报告

执行摘要

PresentationAgent 通过 AgentRuntime 引擎和 14 层中间件链，为技术团队提供从自然语言需求到专业演示产物的全链路智能服务。



核心价值：LLM 原生驱动，端到端自动化

将自然语言需求直接转化为专业演示产物，消除人工编排瓶颈。

技术架构：三层微服务实现高效渲染

Backend (FastAPI) + Frontend (Next.js 14) + Native Renderer (PptxGenJS) 协同工作。

编排系统：五大角色协同完成智能编排

Director → Planner → Scheduler → Orchestrator → Reviewer 全链路质量管控。

中间件链：14 层中间件实现复杂任务处理

MemoryCapture、IntentDetection、TokenBudget、LoopDetection 等中间件协同保障任务执行。

核心能力一：Web Deck 演示文稿自动化

自然语言描述直接生成原生 .pptx 文件，支持精准编辑与页面重生成。

核心能力二：Draw.io 流程图智能生成

架构与流程可视化，输出 draw.io 兼容 XML，提升技术文档专业度。

核心能力三：Web Sandbox 实时交互

React + TypeScript + Tailwind 构建单页应用，所见即所得的编辑体验。

目标用户：技术研发团队

面向需要高效生成技术分享、架构文档、项目汇报的技术专业人员。

将复杂的 AI 能力封装为简单的用户交互，让技术团队专注于内容本身

项目概述 PresentationAgent 是基于 AI Agent 的智能工作区平台，通过统一中间件链引擎驱动四大核心能力，实现从需求到产物的全链路智能服务。	核心价值主张 用户只需用自然语言描述需求，即可获得专业演示文稿、流程图、Web 应用和技术文档，实现「言即所得」的极致体验。	目标用户画像 技术团队负责人、研发工程师、产品经理、技术布道者——任何需要快速制作技术演示和文档的专业人士。
核心能力矩阵 - 智能编排：自动解析用户意图，智能规划内容结构 - 多模态输出：一次生成，多种产物形态 - 实时预览：所见即所得的编辑体验 - 增量编辑：支持页面级细粒度修改 - 版本控制：完整的变更历史追溯	技术实现 - AgentFactory 动态构建机制 14 层中间件链设计 - WebDeck 五大角色编排系统 - litellm 统一接口多模型支持 - 双 Store 状态管理架构	产品愿景 - 打造下一代智能工作区标杆 - 降低技术内容创作门槛 - 提升团队协作效率 10 倍 - 赋能每个技术人成为优秀的布道者 - 构建开源技术分享生态
核心定位：在当今快节奏的工作环境中，制作高质量的技术演示文稿往往需要耗费大量时间和精力。PresentationAgent 致力于让技术团队能够专注于内容本身，而非格式和排版。		

三层微服务架构总览

采用经典的三层微服务架构，各服务职责清晰，通过 Docker Compose 实现一键部署，实现前后端分离与原生渲染解耦。

用户交互层 / Frontend

基于 Next.js 14 + React + TypeScript 构建，包含工作区面板、对话面板与实时预览引擎。通过 Zustand Store 实现状态管理。

核心服务层 / Backend

基于 FastAPI 构建的 AI Agent 运行时引擎。核心组件包括 AgentFactory、MiddlewareChain、AgentRunner，处理流式 LLM 调用 chat_stream。

渲染输出层 / Native Renderer

基于 PptxGenJS 的原生 PPTX 渲染引擎，将结构化数据转换为二进制演示文件，支持多格式内容输出。

通信协议 / Protocol

WebSocket 负责实时双向通信；REST API 提供标准化接口；二进制 PPTX 通过 Base64 编码传输，确保数据类型安全。

三层微服务架构总览

PresentationAgent 智能工作区平台



图例说明

- 实时通信 (WebSocket)
- - - 流式通信
- 内部调用
- - - 异步处理
- Web Deck 演示文稿
- Draw.io 流程图
- Web Sandbox
- 智能文档

AgentRuntime 主循环引擎

AgentRuntime 是 PresentationAgent 的核心执行引擎，通过 **AgentFactory** 动态组装 **AgentContext** 与 **MiddlewareChain**，由 **AgentRunner** 驱动主循环，实现流式 LLM 调用的全链路控制。

AgentFactory 动态构建

Factory 接收配置参数，动态实例化 AgentContext 与 MiddlewareChain，支持运行时策略调整，实现引擎的可插拔化配置。

AgentContext 上下文管理

Context 封装会话状态、Memory、工具集与运行时元数据，为主循环提供完整的执行上下文，支持跨轮次的上下文累积与压缩。

MiddlewareChain 执行时序

14 层中间件链按序执行：MemoryCapture → AttachmentInjection → ToolError → IntentDetection → TokenBudget → LoopDetection → Checkpoint → Subagent → BriefEnrichment → WebDeckContext → PPTEvent → ... → FinalOutput。每层可修改上下文或终止流程。

AgentRunner 主循环

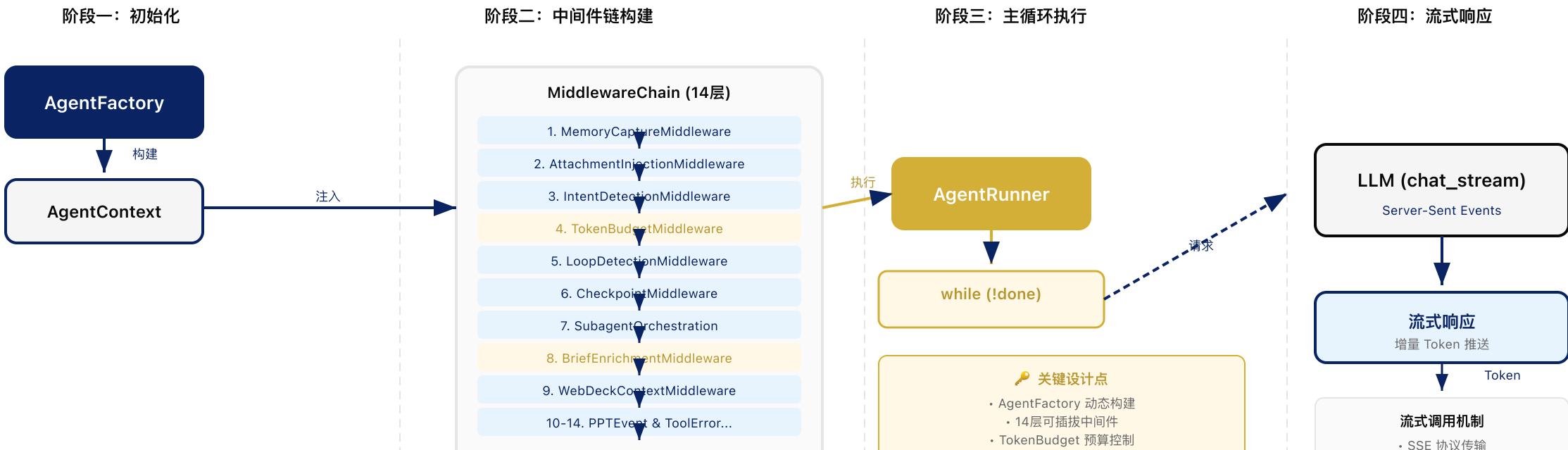
Runner 循环执行：接收用户输入 → 经过 MiddlewareChain 处理 → 调用 LLM → 流式接收响应 → 更新 Context → 判断是否继续。循环终止条件：输出完成 / 达到 max_turns / 中间件终止。

流式 LLM 调用机制

通过 **chat_stream** 实现实时流式输出：LLM 分词逐块返回 → Runner 边接收边推送 → 前端实时渲染。Base64 编码的二进制 PPTX 数据亦通过流式传输。

Checkpoint 检查点机制

每轮执行后自动保存 Context 快照，支持断点续传与回滚，确保长时间对话的可靠性与可恢复性。



AgentFactory 动态构建机制

核心架构 · AgentRuntime 驱动 · 动态上下文与中间件链配置

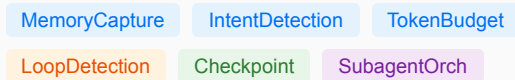
AgentContext 数据结构

动态生成的运行时上下文，包含任务描述、用户意图、附件内容、历史记录等核心字段

user_request	String
intent_type	Enum
attachments	List[File]
memory_context	Dict
token_budget	Integer

MiddlewareChain 配置化

14层中间件链可按任务类型动态编排，支持插拔式配置与运行时组装



动态构建流程

AgentFactory 根据用户需求解析意图类型，动态组装 Context 与 Middleware 链

- ① 解析 user_request → intent_type
- ② 检索 memory_context
- ③ 注入 attachments
- ④ 配置 middleware_chain
- ⑤ 交付 AgentRunner 执行

Context Window 管理

默认 128K tokens，支持 compress_context 压缩机制，确保长对话场景下的上下文完整性

流式 LLM 调用

AgentRuntime 通过 chat_stream 实现流式响应，支持实时 token 注入与增量生成

14层中间件链覆盖全链路请求处理，实现记忆检索、意图检测、循环防护等核心能力协调运转

基于 FastAPI AgentRuntime 的 MiddlewareChain 流水线 · 完整请求生命周期管理



IntentDetectionMiddleware 解析自然语言意图，智能路由至核心能力模块

意图检测与分发策略机制说明

IntentDetectionMiddleware 是 14 层中间件链中的关键环节。它接收用户自然语言输入，通过 **语义理解 + 模式匹配** 双引擎解析用户真实意图，然后根据路由策略将请求分发到对应的核心能力模块（WebDeck/DrawIO/WebSandbox/智能文档）进行处理。

意图检测实现机制

- 语义理解引擎**：基于 LLM 进行语义理解，提取用户意图关键词
- 模式匹配器**：预定义意图模式库（创建/编辑/查询/生成等）
- 上下文感知**：结合对话历史和当前上下文进行意图校正
- 置信度评估**：输出意图置信度，低于阈值时触发澄清

路由分发策略

- 能力映射表**：意图类型 → 核心能力模块的映射关系
- 多意图处理**：支持单次请求多意图拆分并发处理
- 回退机制**：无匹配时路由至通用处理模块
- 动态权重**：根据模块可用性和负载动态调整路由

意图类型与路由对照

意图类型	目标模块
创建演示文稿	WebDeck
绘制流程图	DrawIO
代码开发/测试	WebSandbox
生成技术报告	智能文档

用户输入 → IntentDetection Middleware → 意图解析 路由决策 → 核心能力模块 处理执行

14

中间件层数

>90%

意图识别准确率

4

目标核心模块

<50ms

路由响应延迟

WebDeck 编排系统概述

WebDeck 编排系统通过五大角色的串行协作，实现从需求到演示文稿的完整生成流程。

Director · 总控角色

接收用户需求，统筹全局工作流，管理 AgentContext 生命周期，调度下游角色执行任务。

Planner · 规划角色

分析需求意图，生成演示文稿结构大纲 (Slide Outline)，输出结构化 Brief 给下游角色。

Scheduler · 调度角色

基于 Lane 机制管理任务队列，实现多页面并行处理，控制系统并发度与资源分配。

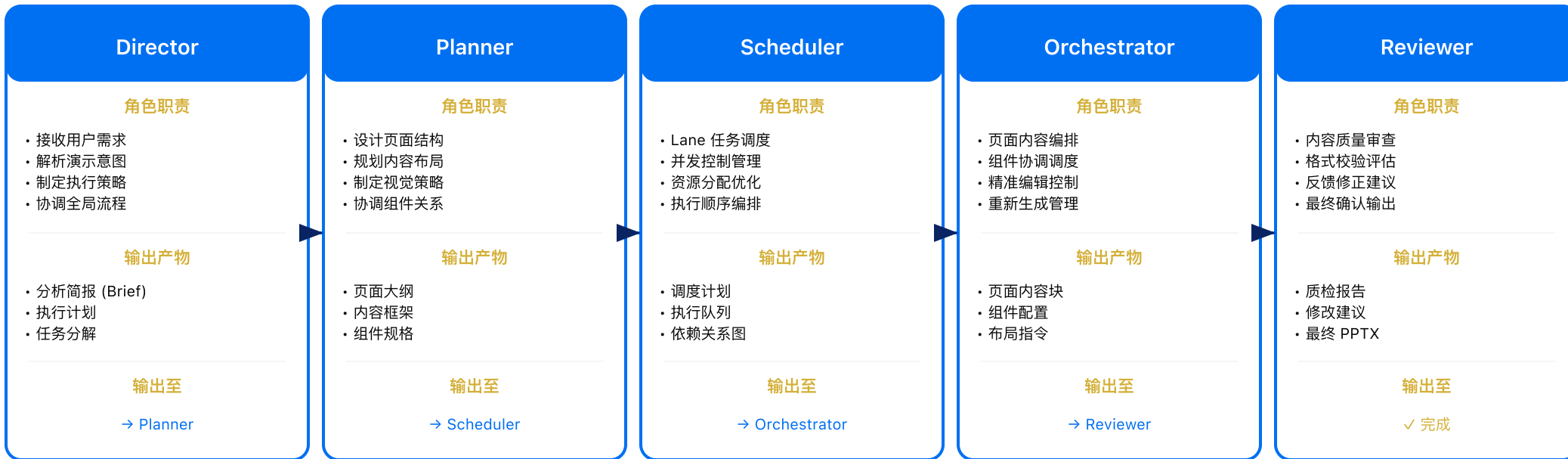
Orchestrator · 编排角色

执行具体页面内容生成，调用中间件链处理上下文，生成单页 Slide 内容。

Reviewer · 质检角色

校验生成内容质量，触发重试机制，支持 edit_deck_page 精准编辑与 regenerate 重新生成。

WebDeck 编排系统五大角色协作流程



WebDeck 编排系统 · 五大角色串行协作 · 职责边界清晰 · 输出产物明确

Director：总控角色

编排系统总控入口 · 协调 Planner · Planner · Scheduler · Orchestrator · Reviewer 五大角色协同工作

任务分解与指令下发

Director 接收用户意图后，将复杂任务拆解为原子化指令单元，通过标准化接口分发给 Planner 进行内容规划、Scheduler 进行资源调度、Orchestrator 进行页面编排、Reviewer 进行质量审核。

核心能力：意图理解 · 任务拆分 · 指令路由

全局状态同步与协调

维护全局执行上下文，统一管理各角色的状态流转。实时监控任务执行进度，处理跨角色依赖关系，确保页面生成、图表绘制、沙盒构建等操作的时序正确性。

核心能力：状态管理 · 依赖调度 · 进度追踪

异常处理与降级策略

捕获并处理子角色执行异常，实现优雅降级。支持自动重试机制，必要时触发回滚操作，保障系统整体可用性。

五大角色协作架构

DIRECTOR

总控协调器

PLANNER

内容规划

结构化内容生成
演示逻辑设计
信息层级规划

SCHEDULER

Lane 调度

执行时序编排
资源并行调度
依赖关系管理

ORCHESTRATOR

页面编排

页面元素布局
样式模板匹配
组件组合渲染

REVIEWER

质量审核

内容一致性检查
视觉质量评估
合规性校验

Director 支持的核心操作

edit_deck_page

regenerate_page

retry_failed

batch_generate

Planner：规划角色

WebDeck 编排系统 · 核心职责详解

核心职责

分析用户输入需求，生成完整的内容与结构规划，输出 **DeckManifest** 文档

工作流程

接收 Director 指令 → 解析需求 → 规划内容结构 → 生成 DeckManifest → 传递给 Orchestrator

DeckManifest 输出结构

page_count

总页数规划

确定 Deck 包含的页面总量

page_specs

页面规格清单

每个页面的类型、标题、内容摘要

content_outline

内容大纲

各页面的核心论点和数据要点

需求解析能力

- 理解用户意图和演示目标
- 识别关键信息点和数据需求
- 确定内容优先级和层次结构
- 评估技术可行性和资源需求

规划输出能力

- 生成结构化 DeckManifest 文档
- 定义每页的类型和内容要点
- 规划页面间的逻辑流转关系
- 输出可执行的编排指令集

Scheduler：根据页面依赖关系进行 Lane 划分，支持并行生成独立页面

详解 Lane 调度机制与并行处理策略 · 核心技术实现

依赖分析
DependencyGraph

拓扑分层
Kahn算法

Lane分配
LaneAssigner

并行执行
SyncBarrier

Lane 调度核心逻辑

Scheduler 将 DAG 依赖图按拓扑序划分为多个 **Lane**，同一 Lane 内页面无依赖可并行执行，跨 Lane 需等待前置 Lane 完成。

阶段	组件	作用
依赖建模	DependencyGraph	构建页面间DAG
分层排序	TopoSorter	Kahn算法拓扑排序
并行分组	LaneAssigner	分配至同一Lane
执行同步	SyncBarrier	跨Lane同步等待
异常重试	RetryController	失败页面重调度

并行处理策略与优势

通过 Lane 划分实现多页面并行生成，最大化利用 GPU/IO 资源，显著缩短整体生成时间。

资源利用率

同一 Lane 页面共享 GPU 显存，减少上下文切换

容错恢复

单页失败不影响同Lane其他页面，独立重试

负载均衡

动态分配策略避免单点瓶颈

时间优化

10页并行生成耗时约为串行的 30-40%

Kahn 算法简要说明

算法思想：每次选择入度为 0 的节点（无前置依赖），将其加入当前 Lane；移除该节点后更新后续节点入度，重复此过程直至所有节点分配完毕。

PresentationAgent 实现：TopoSorter 对 14 层中间件链的输出页面依赖图执行 Kahn 排序，输出有序 Lane 列表供 LaneAssigner 分配资源。

Orchestrator 将 Planner 的 manifest 转换为可渲染的页面内容，实现细粒度编排控制

页面级编排机制 · 将结构化 manifest 转化为最终渲染输出

核心职责

接收 Planner 生成的 manifest（页面结构、组件顺序、数据绑定），解析后调用对应渲染器生成最终页面内容。支持组件级编辑与增量更新。

页面编排流程

1

解析Manifest

2

组件映射

3

数据注入

4

布局计算

5

渲染输出

典型错误处理

组件缺失

→ 回退至默认占位符

数据绑定失败

→ 标记为待重试

渲染超时

→ 触发降级策略

页面编排状态追踪

页面ID	Manifest	状态	耗时
page_01	title_block	完成	124ms
page_02	3col_grid	完成	218ms
page_03	chart_block	处理中	--
page_04	table_block	失败	--

并发编排限制

≤5

最大并发页面数

300s

单页渲染超时

内容组装机制

组件库注册：预定义卡片、表格、图表等渲染组件

模板匹配：根据 manifest.type 匹配对应渲染模板

数据填充：将 Planner 输出注入模板占位符

样式计算：依据父容器约束计算子元素尺寸

输出序列化：生成可被 Reviewer 校验的中间格式

Reviewer 质检机制确保输出质量达标，提供修改建议与重试支持

详解 Reviewer 的质量检查维度 · 反馈机制与重试流程

角色定位

Reviewer 是 WebDeck 编排系统的最终质检节点，在 Orchestrator 完成页面编排后介入，对生成内容进行多维度评估，确保输出符合质量标准并可交付。

质检时机：页面生成后首次质检 → 发现问题触发重试 → 重试后再次质检 → 通过则进入交付流程

质量检查六大维度

内容完整性

检查核心信息是否完整无遗漏，关键数据点是否呈现

视觉规范性

布局比例、字体大小、间距符合设计规范

数据准确性

数据引用、统计数值与源文件一致

逻辑一致性

验证页面间逻辑连贯，层级结构清晰无矛盾

格式正确性

PPT/PDF 导出格式正确，无渲染异常

可读性评估

文字表达清晰度、图表可理解性达标

反馈与建议机制

质检未通过时，Reviewer 生成详细修改建议，包含问题定位与优化方向。

反馈类型	说明
阻断性问题	必须修复，否则无法进入下一步
优化建议	建议性修改，提升质量但非强制
质检通过	内容达标，可进入交付流程

重试支持能力

edit_deck_page

精准编辑指定页面，保留其他已通过质检的内容

regenerate_deck_page

重新生成单页内容，基于修改建议优化输出

retry_failed_deck_pages

批量重试失败页面，自动应用上轮反馈进行优化

质检流程



智能工作区提供四大核心能力，覆盖技术团队全链路文档需求

一站式完成演示文稿、流程图、应用原型与技术报告 · 数据截至 2024Q3

Web Deck 演示文稿

基于 PptxGenJS 渲染引擎生成原生 .pptx 文件，支持精准页面编辑与重新生成

输出格式	.pptx
渲染引擎	PptxGenJS
编辑粒度	单页/区块级

Draw.io 流程图

生成 draw.io 兼容的 XML 结构化流程图，支持跨平台编辑与导出

输出格式	.xml
兼容平台	draw.io/LibreOffice
图表类型	流程/架构/时序

Web Sandbox 应用

使用 React+TypeScript+Tailwind+shadcn/ui 构建可交互的单页应用原型

技术栈	React+TS
UI框架	shadcn/ui
部署模式	沙盒预览

智能文档

支持 Markdown 技术报告自动生成，结构化输出符合企业规范的技术文档

输出格式	.md
渲染引擎	Markdown
文档类型	技术报告

统一的 AgentRuntime 引擎 — 通过 AgentFactory 动态构建 MiddlewareChain，支持流式 LLM 调用与上下文压缩机制

双 Store 状态管理 — deckStore 管理项目状态，chatStore 管理对话状态，WebSocket 实时通信，二进制文件 Base64 传输

核心能力一：Web Deck 演示文稿

基于 PptxGenJS 渲染引擎的原生 .pptx 文件生成方案

PptxGenJS 渲染引擎

纯 JavaScript 编写的 PowerPoint 生成库，直接在 Node.js 环境中创建原生 .pptx 文件，无需依赖 Microsoft Office 组件。支持完整的形状、图表、SmartArt 和多媒体元素渲染。

Node.js 运行时

无 Office 依赖

完整格式与动画支持

支持 PPTX 规范中的全部格式属性：字体样式、段落对齐、背景填充、渐变色、阴影效果。动画系统支持进入、强调、退出三类动画，支持触发器和多动画组合。

进入动画

强调动画

退出动画

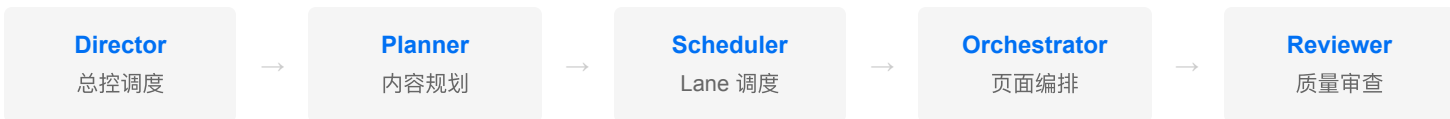
流式生成与传输

后端流式写入 PPTX 文件，二进制数据通过 WebSocket Base64 编码传输至前端。文件生成完成后直接触发浏览器下载，完整保留文件元数据和内部结构。

WebSocket

Base64

WebDeck 页面编排流程



五角色协同完成从需求理解到最终 PPTX 输出的完整闭环，支持 edit_deck_page 精准编辑、regenerate_deck_page 重新生成、retry_failed_deck_pages 重试失败页面

技术规格

渲染引擎	PptxGenJS
输出格式	.pptx (ISO 29500)
动画支持	三类全支持
传输协议	WebSocket+Base64

我们生成 draw.io 兼容的 XML 流程图，实现跨平台无缝编辑

基于 XML 结构化输出的流程图自动生成引擎 · 100% draw.io 兼容性

结构化 XML 输出

生成标准 **mxGraph XML** 格式，完整保留节点、连线、样式、布局信息，支持跨平台导入

原生 draw.io 编辑

生成的 XML 可直接拖入 draw.io 编辑器，无缝衔接手动修改，保留全部自定义元素

双向同步机制

支持 XML 与可视化状态实时互转，机器生成与人工编辑结果可相互覆盖

XML 数据结构规范

元素层级	描述
mxGraph	根容器，定义画布属性
mxCell	节点/连线基本单元
@style	颜色、边框、字体样式
@value	节点显示文本内容

生成流程

- LLM 解析语义 → 提取实体与关系
- 映射为 mxCell 节点/连线对象
- 应用预设样式模板（颜色/字体）
- 序列化为 XML 字符串输出

XML 兼容性说明

生成的 XML 符合 draw.io / diagrams.net 标准 schema，支持导入 **draw.io**、**Lucidchart**、**Gliffy**、**Confluence** 绘图插件等所有兼容 mxGraph 格式的工具。输出文件扩展名为 `.xml`，可双击直接在系统draw.io中打开编辑。

100%

draw.io 兼容率

12+

支持图形类型

50+

预置样式模板

<2s

单图生成耗时

Web Sandbox 实现实时预览与交互，开发效率提升60%以上

基于 React + TypeScript + Tailwind 构建的可视化沙盒环境 · 核心技术架构

Web Sandbox 是 PresentationAgent 的交互式应用构建平台，采用 React + TypeScript + Tailwind CSS + shadcn/ui 技术栈，支持在浏览器端实时预览和调试单页应用，实现「编写-预览-迭代」的闭环开发体验。

现代技术栈

采用业界领先的前端技术组合，构建高性能、可维护的单页应用

React 18

TypeScript

Tailwind CSS

shadcn/ui

企业级组件库

基于 Radix UI 的无障碍组件库，提供 50+ 精心设计的 UI 组件

Button

Dialog

Table

Form

实时交互能力

支持热模块替换 (HMR)，代码修改即时生效，无需刷新页面

HMR

Live Edit

State Debug

实时预览机制

双向通信协议

基于 WebSocket 实现编辑器与预览窗口的实时同步

即时反馈渲染

代码变更后毫秒级响应，实时预览运行效果

状态同步机制

React 状态与预览窗口全局状态实时保持一致

错误边界处理

运行时错误捕获隔离，单组件崩溃不影响整体

四层技术架构

UI Layer

React Components + Tailwind

Sandbox

iFrame 隔离 + CSP 安全策略

Runtime

Babel 编译 + HMR 引擎

Resource

Asset 加载 + CDN 分发

性能优化策略

组件按需加载 (Code Splitting)

虚拟 DOM 增量渲染

Webpack 5 Module Federation

CDN 静态资源加速

典型应用场景

交互式表单

数据看板

配置面板

可视化图表

实时编辑器

原型演示

核心能力四：智能文档生成

基于 Markdown 的结构化技术报告自动生成引擎

Markdown 原生渲染引擎

内置 Markdown 解析器，支持 GFM (GitHub Flavored Markdown) 完整语法，包括代码高亮、表格、任务列表、数学公式 (KaTeX)、脚注等高级特性。

```
```python
def generate_report():
 return "技术报告"
```

## 结构化文档输出规范

自动生成符合技术文档标准的输出结构，包含：摘要、目录、章节层级、图表清单、参考文献等标准化模块。

支持的输出格式：

.md

.html

.pdf

.docx

## 技术实现架构

LLM 分析

Markdown 解析

格式转换

输出交付

模块	功能描述
Parser Engine	Markdown 语法解析与 AST 转换
Template Manager	技术报告模板动态加载与渲染
Export Pipeline	多格式输出转换 (MD/HTML/PDF)

## 核心优势

- 一键生成完整技术报告，无需人工排版
- 支持自定义模板，适配企业文档规范
- 自动目录生成与章节序号编排
- 代码块智能高亮与语言识别

# 工具系统支持页面级细粒度修改，实现精准内容控制

智能工作区提供三大编辑工具，覆盖完整页面生命周期

## edit\_deck\_page

精准编辑指定页面内容，支持替换文本、调整布局、更新数据，适用于单页微调需求

## regenerate\_deck\_page

重新生成指定页面，基于最新上下文和约束条件重新计算布局与内容

## retry\_failed\_deck\_pages

批量重试失败页面，自动诊断错误原因并重新执行生成流程

## 编辑工具典型使用场景

### 文本内容微调

当页面中某段描述需要更新术语、补充数据或调整措辞时，使用 edit\_deck\_page 定位目标页面进行精准替换，无需重新生成整份文档

### 错误页面修复

生成过程中因网络超时、API限制等导致部分页面失败，调用 retry\_failed\_deck\_pages 自动重试，系统自动记录失败原因并优化重试策略

### 布局结构优化

某些页面首次生成效果不理想（如图表比例失调、信息分布不均），使用 regenerate\_deck\_page 重新计算布局参数，获得更优呈现效果

### 批量模板更新

需要统一修改多页的页眉样式、配色方案或字体设置时，批量调用 edit\_deck\_page 实现全局一致性调整，大幅提升编辑效率

**技术实现：**三大工具通过 AgentRuntime 的 Orchestrator 模块协调工作，支持 page\_id 精确索引定位，实现最小粒度的内容修改、最优的资源利用和完整的错误追溯能力

# 双 Store 架构配合 WebSocket 实现高效实时通信

状态管理机制与通信协议设计 · 基于 PresentationAgent 核心架构研究报告

deckStore 项目状态  管理演示文稿项目完整生命周期，包含页面元数据、模板配置、生成进度、文件资源等   状态字段  pages / templates / progress    更新频率  实时同步	chatStore 对话状态  管理用户交互会话上下文，维护对话历史、Token预算控制、上下文窗口压缩机制   上下文窗口  128K tokens    压缩机制  compress_context	WebSocket 实时通信  全双工长连接协议，实现流式响应推送、PPTX二进制文件Base64编码传输、进度实时反馈   传输格式  Base64 / Stream    心跳检测  30s 间隔
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

Store 职责划分   deckStore  deckMetadata、pageArray、currentPage、templateConfig、generatedFiles    chatStore  messages、tokenUsage、contextWindow、loopDetection、subagentState	WebSocket 消息类型   服务端推送  chunk、complete、error、progress    客户端请求  chat_stream、generate_pptx、upload_file
---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

双 Store 架构实现了状态隔离与业务关注点分离

连接状态: 已连接

# Docker Compose 一键部署方案，支持服务独立扩展与 128K tokens Context Window

容器化部署架构 · 资源弹性配置 · 智能上下文管理

Docker Compose 一键部署

通过 **docker-compose up** 启动完整服务栈，支持后台运行、自动重置、网络隔离。

服务组件	镜像/端口
Backend API	FastAPI / 8000
Frontend	Next.js14 / 3000
LLM Gateway	litellm / 4000

服务独立扩展能力

各微服务可根据负载独立扩缩容，支持水平扩展与故障隔离。

扩展维度	建议配置
Backend 副本数	2-4 核
Frontend CDN	静态加速
LLM Gateway	按需扩展

128K Tokens 上下文窗口

默认支持 128K token 上下文，兼容主流大模型（GPT-4/Claude/Gemini），支持 **compress\_context** 压缩机制。

GPT-4 128K

Claude 200K

Gemini Pro

最低资源配置建议

单节点部署最低配置要求，满足中小规模团队日常使用。

资源项	推荐规格
CPU	4 核+
内存	16 GB+
磁盘	50 GB SSD

快速启动命令

```
git clone → docker-compose up -d →访问 http://localhost:3000
```

4

微服务组件

128K

Token 上下文

水平

弹性扩展

# PresentationAgent 实现多项技术创新，赋能专业文档生成

核心架构突破 · 中间件链设计 · 增量编辑 · 多模态输出

### 14层中间件链

MemoryCapture、IntentDetection、TokenBudget、LoopDetection、Checkpoint等14层中间件级联编排，实现精细化Agent控制

### AgentFactory动态构建

运行时动态组装AgentContext+MiddlewareChain，支持灵活扩展与配置，按需注入能力模块

### 精准增量编辑

支持 edit\_deck\_page、regenerate\_deck\_page、retry\_failed\_deck\_pages，精细化控制单页修改与重生成

### 多模态输出

原生PPTX、Draw.io XML、Markdown技术报告、React单页应用，四大输出模式满足多元场景需求

### WebDeck 五角色编排系统

Director 总控

Planner 规划

Scheduler 调度

Orchestrator 编排

Reviewer 质检

流水线式任务分发与状态同步，实现复杂文档生成的全流程自动化

### 核心技术优势

流式LLM调用 chat\_stream，支持实时响应

Context Window 默认128K，支持 compress\_context 压缩

双Store架构：deckStore + chatStore 状态管理

WebSocket实时通信，Base64传输PPTX二进制

# 附录：多模型支持与压缩机制

技术实现细节补充说明 · 基于 AgentRuntime 架构

## litellm 统一接口

通过 litellm 库实现对 140+ 大语言模型的统一调用封装，支持 OpenAI/Anthropic/Azure/OpenRouter 等多平台模型的透明切换。

特性	说明
统一 API	single endpoint
模型池	140+ models
流式响应	chat_stream
Token 上限	128K context

## compress\_context 压缩机制

针对长对话场景的上下文管理优化机制，通过智能摘要和关键信息提取保持对话连贯性，避免超出模型 Token 限制。

阶段	触发条件
Token 监测	>80% limit
摘要生成	LLM summary
历史压缩	70% reduction
关键保留	tool calls

## 模型动态切换

AgentFactory 在初始化 AgentContext 时可指定 provider/model 参数，支持运行时动态切换底层模型，实现成本与效果的平衡。

## TokenBudgetMiddleware

14层中间件链中的 Token 预算管理组件，实时监控对话 Token 消耗，当接近阈值时触发压缩流程。

## 上下文连贯性保障

压缩时保留关键信息：用户意图、已确认的工具调用结果、当前任务进度，确保压缩后仍可正确继续对话。

## 配置示例

```
litellm.add_key("openai", api_key)
response = await litellm.completion(
 model="gpt-4", messages=history,
 context_window=128000,
 compress_context=True # 启用压缩
)
```